Informe Post-Incidente y Recuperación ante Desastres — Unidad 6

Proyecto: Food Store (PostgreSQL 16+) | Bases de Datos II **Equipo:** Máximo, Andrés y Octavio **Fecha:** 04/11/2026

1. Datos Generales del Respaldo Utilizado

Archivo Dump: `backups/foodstore_20261104_164552.dump` | **Formato:** Custom binario comprimido (`pg_dump -Fc`) | **Entradas TOC:** 58

Fecha/Hora de Creación: 2026-11-04 16:45:54 | **Tamaño:** 4.133.600 bytes (3,94 MB) | **Volumen Origen:** 891.992 registros totales

2. Métricas de Recuperación ante Desastres (RTO y RPO)

Métrica	Definición Operativa en el Simulacro	Valor Previsto (SLA)	Valor Observado (Real)	Evaluación
RTO (Recovery Time Objective)	Cronómetro desde `createdb`, ejecución de `pg_restore` hasta finalización de validaciones lógicas.	< 300,00 s (5 min)	**6,95 segundos**	**Cumple con holgura (-97,7%)**
RPO (Recovery Point Objective)	Antigüedad del respaldo utilizado al momento de la declaración del desastre (pérdida potencial).	≤ 24,00 horas (Diario)	**332,3 segundos (~5,54 min)**	**Cumple ventana diaria**

3. Resultado de Validaciones de Integridad y Restricciones Lógicas

Tabla / Entidad	Filas en Origen (`foodstore`)	Filas Restauradas (`foodstore_restore_test`)	Controles Lógicos de Negocio	Resultado
`categoria`	5	5	Categorías activas y consistentes	**100% Coincidencia**
`cliente` (vista sobre `usuario`)	20.000	20.000	Cuentas e integridad de mail único	**100% Coincidencia**
`producto`	50.000	50.000	`COUNT(*) WHERE precio < 0 OR stock < 0` = **0**	**100% Coincidencia**
`pedido`	200.000	200.000	Estados válidos y FK referencial estricta	**100% Coincidencia**
`detalle_pedido`	621.987	621.987	`COUNT(*) WHERE cantidad ≤ 0 OR precio_unitario < 0` = **0**	**100% Coincidencia**

4. Problemas Reales Encontrados

- **Codificación ANSI en PowerShell 5.1:** Caracteres UTF-8 sin BOM (guion largo –, bytes `E2 80 94`) hacían que el byte `0x94` se interpretara como comilla de cierre ” desbalanceando el parser. Se normalizó a caracteres estándar.
- **Discrepancia nominal de entidad:** La cátedra consulta `cliente`, pero la tabla canónica es `usuario`. Se solucionó arquitectónicamente con la vista `cliente`.

5. Mejora Concreta para la Próxima Ejecución

- **Rotación automática y multihilo:** Implementar política de retención de últimos 7 respaldos con purga automática en `backup_foodstore.ps1`.
- **Paralelismo en restauración:** Agregar flag `-j 4` a `pg_restore` para acelerar la creación paralela de índices en servidores multinúcleo.

6. Asistencia con Inteligencia Artificial (OpenCode/Kiro) y Verificación Humana

Intervención de IA: Kiro y OpenCode generaron la especificación inicial, el script de backup y una propuesta de mantenimiento que sugería `VACUUM FULL`.
Corrección y decisión humana: El equipo descartó categóricamente `VACUUM FULL` por su bloqueo exclusivo total (`AccessExclusiveLock`) que causa caída total de lecturas y escrituras, reemplazándolo por `VACUUM ANALYZE` concurrente (`ShareUpdateExclusiveLock`). Asimismo, se aseguró la captura inmediata de `$LASTEXITCODE` en PowerShell y los flags `--no-owner --no-privileges`.

Verificación: Ejecución real en PostgreSQL 17.10 con cronómetro de precisión, log generado y verificación con EXCEPT de 0 discrepancias.
